

Практическая работа №3. Распределенная модель обработки данных

1. Назначение практической работы

Цель работы — освоить базовый цикл распределённой обработки табличных данных средствами Apache Spark и PySpark. В данной практике рассматривается как запустить локальное Spark-приложение, подготовить исходные таблицы из базы SQLite, прочитать их в DataFrame, исследовать и задать схему, выполнить преобразования, корректно обработать пропуски и типы, сформировать агрегаты, соединить связанные таблицы, повторить аналитическую задачу через Spark SQL и сохранить итоговый набор данных. Акцент делается не на механическом переносе синтаксиса Pandas в PySpark, а на понимании того, что DataFrame Spark является частью лениво вычисляемого распределённого плана.

Практика выполняется в локальном режиме Spark. Такой режим использует ресурсы одного компьютера и не превращает его в физический кластер, однако сохраняет программную модель Spark, в котором данные разбиваются на разделы, операции над DataFrame формируют план вычислений, а действия запускают выполнение задач. Поэтому один и тот же код после адаптации источников и параметров может быть перенесён на кластерную инфраструктуру.

Ожидаемые результаты обучения

- создавать SparkSession и объяснять назначение Driver, Executor и режима local[*];
- различать transformation и action, понимать принцип lazy evaluation;
- читать табличные данные в Spark DataFrame и анализировать printSchema();
- сравнивать автоматическое определение схемы с явным StructType;
- применять select(), filter(), withColumn(), drop(), distinct()/dropDuplicates() и orderBy();
- обнаруживать пропуски, объяснять их предметный смысл и выполнять обоснованные операции заполнения или фильтрации;
- преобразовывать строковые даты и числовые поля в типы, пригодные для вычислений;
- выполнять groupBy().agg() и интерпретировать агрегаты;
- соединять DataFrame через join() и контролировать сохранение строк при разных типах соединения;
- создавать временные представления и выполнять аналитические запросы через spark.sql();

- записывать результат в CSV и Parquet и понимать, почему Spark создаёт набор part-файлов.

Исходные данные

В работе используется предоставленная база данных ecommerce.sqlite, в пользовательской части базы содержится 11 таблиц. Кроме них SQLite хранит служебные таблицы статистики и последовательностей.

Практика сосредоточена на транзакционном контуре интернет-магазина: заказах, позициях заказов, товарах, клиентах, платежах, отзывах и справочнике перевода категорий. Таблица geolocation содержит более одного миллиона строк, но в основной последовательности работы не используется, чтобы практика оставалась выполнимой на типовом ноутбуке.

Таблица 1. Таблицы ecommerce, используемые в практической работе

Таблица	Строк	Назначение
orders	99 441	Заказы, статусы и временные отметки.
order_items	112 650	Позиции заказов, товары, продавцы, цена и стоимость доставки.
products	32 951	Категория товара и физические характеристики.
customers	99 441	Идентификатор клиента, город и штат.
order_payments	103 886	Способ оплаты, число платежей/рассрочек и сумма.
order_reviews	97 621	Оценка и текст отзыва; используется для анализа качества данных и будущих NLP-задач.
product_category_name_translation	71	Соответствие исходного имени категории английскому названию.

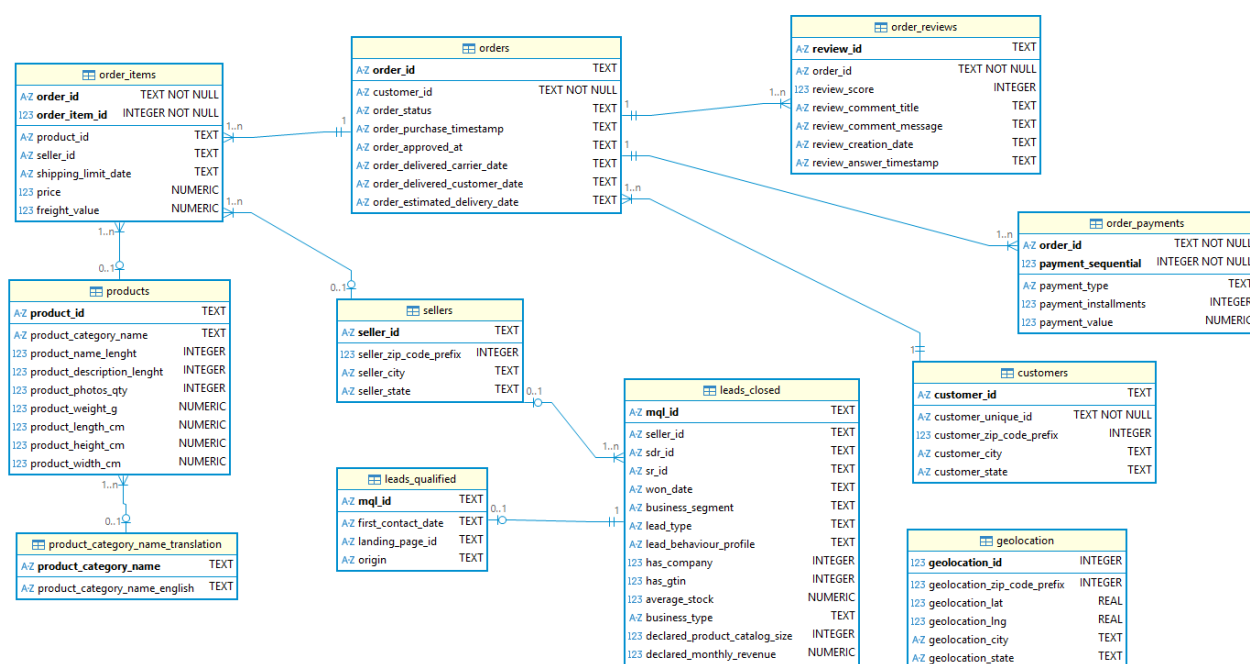


Рис. 1. Схема связей таблиц базы ecommerce.sqlite

Схема показывает, что `order_items` является центральной таблицей фактов на уровне позиции заказа. Она связана с `orders`, `products` и `sellers`. Таблица `orders`, в свою очередь, связывает транзакцию с `customers`, `payments` и `reviews`. Такое устройство делает набор удобным для демонстрации распределённых JOIN и построения аналитических витрин.

2. Распределенная модель обработки данных в Apache Spark

Apache Spark — вычислительный движок для параллельной обработки данных. Его программная модель отделяет описание аналитической операции от непосредственного выполнения. Пользователь строит цепочку преобразований `DataFrame`, Spark формирует логический и физический планы, а затем исполняет их по разделам данных. При кластерном запуске разделы обрабатываются на разных вычислительных узлах. В локальном исполнении `local[*]` та же модель воспроизводится потоками одного узла.

2.1. Driver, Executor и разделы данных

Driver — управляющая часть Spark-приложения. В нём создаётся `SparkSession`, строится план вычислений и координируется выполнение. Executor — вычислительная часть, выполняющая задачи над разделами данных и хранящая промежуточные результаты. В физическом кластере executors обычно работают на отдельных узлах. В локальном режиме их роль эмулируется внутри одного процесса JVM, поэтому локальный запуск полезен для обучения и отладки, но не является тестом реальной сетевой масштабируемости.

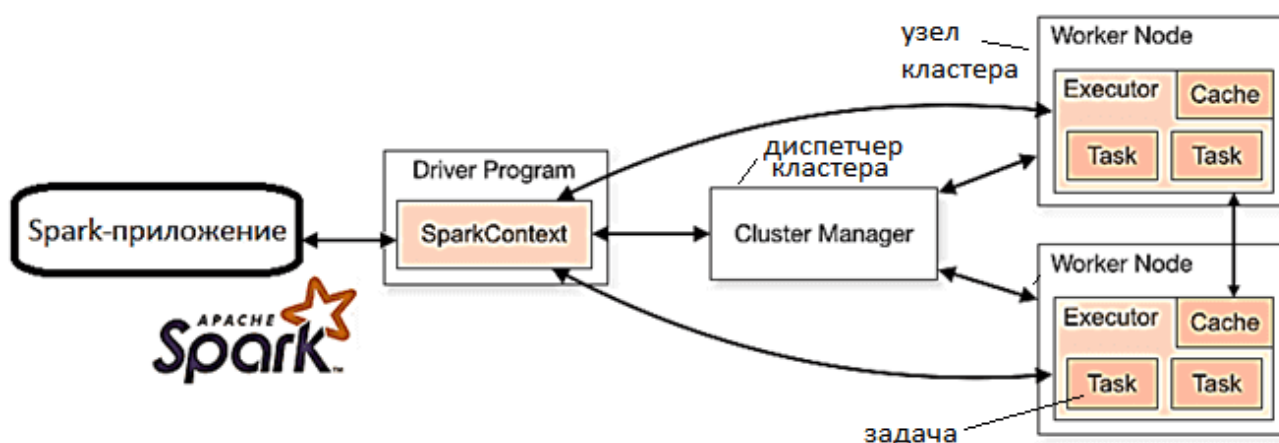


Рис. 2. Архитектура Spark приложения

Набор данных Spark логически делится на `partitions` $P_1, P_2, \dots, P_k$. Минимальная единица планируемой работы — `task`, обычно обрабатывающая один `partition` на конкретной стадии. Число разделов влияет на параллелизм и

накладные расходы. Слишком мало разделов ограничивает параллелизм, слишком много создаёт большое число мелких задач.

2.2. DataFrame, transformations и actions

Spark DataFrame — распределённая табличная структура с именованными столбцами и схемой типов. В отличие от Pandas DataFrame, объект PySpark обычно не означает, что все строки уже загружены в память Python-процесса. Выражение `select()`, `filter()`, `withColumn()`, `groupBy()` или `join()` создаёт новое описание данных и относится к transformations. Методы `count()`, `show()`, `collect()` и запись на диск относятся к actions: именно они требуют получить результат и тем самым запускают вычисление.

2.3. Lazy evaluation

Принцип lazy evaluation означает, что Spark откладывает вычисление цепочки преобразований до появления необходимости что-то рассчитать (action). Это позволяет оптимизатору объединять операции, удалять лишние вычисления и выбирать физические стратегии. Следовательно, время создания переменной `df2 = df.filter(...)` почти ничего не говорит о стоимости обработки. Реальные чтение, фильтрация, обмен данными и агрегация происходят при action.

В отчёте важно различать «создан DataFrame» и «данные вычислены». Если код содержит только преобразования (transformations), результат ещё может не быть материализован. Функции `show()`, `count()` и `write` являются удобными точками, в которых можно наблюдать фактический запуск вычислительного плана.

3. Рабочее окружение и подготовка источника

Для учебной среды рекомендуется единый интерпретатор Python 3.11 и JDK 17 или 21. Такая фиксация уменьшает вероятность несовместимости между версиями Python, Java и PySpark. Отдельная установка Hadoop для локальной работы с DataFrame не требуется. PySpark устанавливается как Python-пакет и запускает локальную JVM через Java, поэтому команда `java -version` должна выполняться до запуска notebook.

Практический блок 3.1. Подготовка окружения

Команда `py -3.11` выбирает интерпретатор Python 3.11, модуль `venv` создаёт изолированное окружение, а вызов `python -m pip` привязывает установку пакетов именно к активному интерпретатору. Пакет `pyspark` содержит Python API и компоненты Spark, `pandas` используется только для лёгкого извлечения таблиц из

SQLite в промежуточные CSV, jupyter — для интерактивного выполнения. Команда `java -version` проверяет наличие JDK.

```
# Windows PowerShell / cmd
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install pyspark pandas jupyter

java -version
python -c "import pyspark; print(pyspark.__version__)"

jupyter notebook
```

В отчёте следует зафиксировать фактические версии Python, Java и PySpark. Номер версии PySpark намеренно не зашивается в методичку, он зависит от используемого преподавателем или студентом окружения.

Окружение подготовлено корректно, если `java -version` завершается без ошибки, `import pyspark` выполняется в активированной `.venv`, а Jupyter запускается из того же окружения. При диагностике проблем с Spark сначала проверяют версии Python и Java, затем переменную `JAVA_HOME`, и только после этого код `DataFrame`.

Практический блок 3.2. Извлечение таблиц SQLite в staging-каталог

Spark не читает файл SQLite как встроенный табличный формат подобно CSV или Parquet. Прямое подключение возможно через JDBC-драйвер SQLite, однако тогда учебная работа зависит от внешнего JAR-файла и его версии.

В основной траектории используется воспроизводимый staging-этап: стандартный модуль `sqlite3` открывает базу, `pandas` выполняет `SELECT *` для выбранной таблицы, а `to_csv()` создаёт файл, который затем читает Spark. Это отделяет слой источника данных от слоя распределённой обработки.

`Path` формирует переносимые пути, `connect()` открывает SQLite-базу, контекстный менеджер `with` гарантирует закрытие соединения. `read_sql_query()` возвращает `Pandas DataFrame`, а `to_csv(index=False)` не сохраняет технический индекс. Цикл выполняет одинаковую операцию для всех выбранных таблиц и печатает контрольное число строк.

```
from pathlib import Path
import sqlite3
import pandas as pd
```

```

DB_PATH = Path("ecommerce.sqlite")
STAGING = Path("spark_input")
STAGING.mkdir(exist_ok=True)

TABLES = [
    "orders",
    "order_items",
    "products",
    "customers",
    "order_payments",
    "order_reviews",
    "product_category_name_translation",
]

with sqlite3.connect(DB_PATH) as conn:
    for table in TABLES:
        part = pd.read_sql_query(
            f'SELECT * FROM "{table}"',
            conn
        )
        part.to_csv(
            STAGING / f"{table}.csv",
            index=False,
            encoding="utf-8"
        )
        print(f"{table:36s} {len(part):>8} rows")

```

Результат выполнения

orders	99441 rows
order_items	112650 rows
products	32951 rows
customers	99441 rows
order_payments	103886 rows
order_reviews	97621 rows
product_category_name_translation	71 rows

Staging не меняет содержательную структуру таблиц, он только переводит выбранные отношения SQLite в формат, который Spark умеет читать без внешнего JDBC-драйвера. Самая крупная из используемых таблиц — order_items со 112 650 строками. Полный файл geolocation сознательно не извлекается в основной части работы.

4. Запуск локального Spark

Точка входа для работы с DataFrame и Spark SQL — SparkSession. Builder накапливает конфигурацию приложения, appName задаёт понятное имя, master("local[*]") разрешает Spark использовать доступные логические ядра локального компьютера. Параметр spark.sql.shuffle.partitions задаёт число разделов, используемых по умолчанию после перераспределяющих SQL-

операций; для небольшого учебного набора значение 8 уменьшает число мелких задач по сравнению с серверными настройками по умолчанию.

Практический блок 4.1. Создание SparkSession

Метод `getOrCreate()` создаёт новую сессию либо возвращает уже существующую. `spark.sparkContext` представляет низкоуровневый контекст приложения; `setLogLevel("WARN")` уменьшает объём служебных сообщений в `notebook`. Атрибут `master` должен показать `local[*]`, а `appName` — заданное имя. Номер версии Spark зависит от установленного пакета и поэтому выводится студентом как часть отчёта, а не фиксируется в ожидаемом результате.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (
    SparkSession.builder
    .appName("Practice4_Ecommerce")
    .master("local[*]")
    .config("spark.driver.memory", "2g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

print("Master:", spark.sparkContext.master)
print("App:", spark.sparkContext.appName)
print("Spark:", spark.version)
```

Результат выполнения

```
Master: local[*]
App: Practice4_Ecommerce
Spark: <версия из локального окружения>
```

`local[*]` означает использование одного компьютера, а не реального кластера. При этом API `DataFrame`, ленивое планирование и разбиение обработки на задачи сохраняются. Поэтому практика формирует правильную модель программирования Spark, не требуя кластерной инфраструктуры.

5. Чтение данных и работа со схемой

Схема `DataFrame` определяет имя и тип каждого столбца. Для CSV схема не хранится внутри формата. Spark либо читает все значения как строки, либо выполняет дополнительный анализ для `inferSchema`, либо получает `StructType` от программиста.

Для аналитического потока обработки явная схема обычно предпочтительнее. Она воспроизводима, не требует повторного прохода для вывода типов и фиксирует контракт входных данных.

Практический блок 5.1. Чтение orders без автоматического вывода типов

Метод `spark.read` создаёт объект `DataFrameReader`. Метод `option("header", True)` сообщает, что первая строка содержит имена столбцов. Если `inferSchema` не задан, CSV-значения читаются как `string`.

Метод `printSchema()` печатает дерево схемы, `count()` является action и запускает подсчёт строк, `select().show()` выводит небольшой срез без переноса всей таблицы в Python. Параметр `truncate=False` сохраняет полные идентификаторы.

```
orders_raw = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .csv(str(STAGING / "orders.csv"))
)

orders_raw.printSchema()
print("Строк:", orders_raw.count())

orders_raw.select(
    "order_id",
    "customer_id",
    "order_status",
    "order_purchase_timestamp"
).show(5, truncate=False)
```

Результат выполнения

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_status: string (nullable = true)
|-- order_purchase_timestamp: string (nullable = true)
|-- order_approved_at: string (nullable = true)
|-- order_delivered_carrier_date: string (nullable = true)
|-- order_delivered_customer_date: string (nullable = true)
|-- order_estimated_delivery_date: string (nullable = true)

Строк: 99441
```

Таблица 2. Первые пять строк orders после чтения CSV

order_id	customer_id	status	purchase_timestamp
e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33
53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37
47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49
949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06
ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39

orders содержит 99 441 строк, но все поля после чтения CSV имеют тип string. Для идентификаторов это корректно, а для временных отметок — нет. Вычислять длительность доставки и извлекать год/месяц следует только после преобразования в timestamp.

Практический блок 5.2. Явная схема для order_items

StructType описывает схему таблицы как список StructField. Каждый StructField связывает имя столбца с типом данных и признаком nullable. StringType используется для идентификаторов, IntegerType — для номера позиции, DoubleType — для цены и стоимости доставки. При чтении reader.schema(schema) Spark использует этот контракт вместо автоматического вывода типов.

```
from pyspark.sql.types import (
    StructType, StructField,
    StringType, IntegerType, DoubleType
)

order_items_schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("order_item_id", IntegerType(), True),
    StructField("product_id", StringType(), True),
    StructField("seller_id", StringType(), True),
    StructField("shipping_limit_date", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("freight_value", DoubleType(), True),
])

items = (
    spark.read
    .option("header", True)
    .schema(order_items_schema)
    .csv(str(STAGING / "order_items.csv"))
)

items.printSchema()
print("Строк:", items.count())
```

Результат выполнения

```
root
|-- order_id: string (nullable = true)
|-- order_item_id: integer (nullable = true)
|-- product_id: string (nullable = true)
|-- seller_id: string (nullable = true)
|-- shipping_limit_date: string (nullable = true)
|-- price: double (nullable = true)
|-- freight_value: double (nullable = true)

Строк: 112650
```

Явная схема сразу делает `price` и `freight_value` числовыми. Это устраняет неоднозначность CSV и позволяет выполнять арифметику без дополнительного приведения в каждом выражении. Строковым оставлен `shipping_limit_date`, его можно преобразовать в `timestamp` только тогда, когда временная семантика действительно нужна.

Практический блок 5.3. Преобразование временных полей `orders`

Функция `F.to_timestamp(column, format)` преобразует строковое значение в Spark SQL `TimestampType`. Цикл применяет одно правило к пяти временным атрибутам. `withColumn()` создаёт новый `DataFrame`, заменяя одноимённый столбец результатом выражения. Исходный `orders_raw` при этом не изменяется: `DataFrame` Spark концептуально иммутабелен.

```
timestamp_cols = [
    "order_purchase_timestamp",
    "order_approved_at",
    "order_delivered_carrier_date",
    "order_delivered_customer_date",
    "order_estimated_delivery_date",
]

orders = orders_raw
for c in timestamp_cols:
    orders = orders.withColumn(
        c,
        F.to_timestamp(F.col(c), "yyyy-MM-dd HH:mm:ss")
    )

orders.printSchema()
```

Результат выполнения

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_status: string (nullable = true)
|-- order_purchase_timestamp: timestamp (nullable = true)
|-- order_approved_at: timestamp (nullable = true)
|-- order_delivered_carrier_date: timestamp (nullable = true)
|-- order_delivered_customer_date: timestamp (nullable = true)
|-- order_estimated_delivery_date: timestamp (nullable = true)
```

После преобразования Spark понимает временную семантику пяти полей. Сравнение строк дат может случайно работать при ISO-формате, но операции `datediff()`, `year()`, `month()` и `date_format()` должны выполняться над типами `date/timestamp`, а не над текстом.

6. Основные преобразования DataFrame

Преобразования DataFrame описывают новый набор строк и столбцов без изменения исходного объекта. Метод `select()` ограничивает набор полей, `filter()` формирует условный срез, `withColumn()` создаёт или заменяет столбец, `drop()` исключает поле, `orderBy()` задаёт порядок только для итогового представления, а `distinct()/dropDuplicates()` удаляют повторяющиеся значения. В распределённой системе порядок строк без явного `orderBy` не является частью гарантированного результата.

Практический блок 6.1. Фильтрация и вычисляемый признак стоимости позиции

В следующем коде отбираются позиции с $\text{price} \geq 500$. Выражение

`F.col("price") >= 500`

создаёт условие Spark SQL.

Метод `withColumn("gross_item_value", ...)` добавляет цену с доставкой, `F.round(..., 2)` округляет результат, `select()` оставляет только аналитически необходимые поля, а `orderBy(F.desc(...))` делает показ первых строк детерминированным и одновременно показывает самые дорогие позиции.

Метод `count()` запускает вычисление всего фильтра.

```
expensive_items = (
    items
    .filter(F.col("price") >= 500)
    .withColumn(
        "gross_item_value",
        F.round(F.col("price") + F.col("freight_value"), 2)
    )
    .select(
        "order_id", "order_item_id",
        "price", "freight_value", "gross_item_value"
    )
)

print("Позиций с price >= 500:", expensive_items.count())

expensive_items.orderBy(
    F.desc("gross_item_value")
).show(5, truncate=False)
```

Результат выполнения

```
Позиций с price >= 500: 3239
```

Таблица 3. Пять позиций с максимальным $price + freight_value$ среди $price \geq 500$

order_id	item	price	freight	gross
0812eb902a67711a1cb742b3cdaa65ae	1	6735.00	194.31	6929.31
fefacc66af859508bf1a7934eab1e97f	1	6729.00	193.21	6922.21
f5136e38d1a14a4dbd87dff67da82701	1	6499.00	227.66	6726.66
a96610ab360d42a2e5335a3998b4718a	1	4799.00	151.34	4950.34
199af31afc78c699f0dbf71fb178d4d4	1	4690.00	74.34	4764.34

Условию $price \geq 500$ соответствуют 3 239 из 112 650 позиций. `gross_item_value` является вычисляемым признаком уровня позиции, а не стоимостью всего заказа: один `order_id` может иметь несколько `order_item_id`. Этот пример показывает, почему единица наблюдения должна быть определена до интерпретации результата.

Практический блок 6.2. Временные преобразования заказов

Для доставленных заказов формируется компактный аналитический набор. Метод `filter()` оставляет `order_status == "delivered"`. Метод `date_format()` создаёт календарный месяц покупки, `datediff(end, start)` считает число календарных дней между датами, а сравнение фактической даты доставки с `estimated_delivery_date` создаёт булев индикатор `is_late`. Проверки `isNotNull()` используется отдельно, чтобы отличить отсутствие даты доставки от факта просрочки.

```
delivered = (
    orders
    .filter(F.col("order_status") == "delivered")
    .withColumn(
        "purchase_month",
        F.date_format("order_purchase_timestamp", "yyyy-MM")
    )
    .withColumn(
        "delivery_days",
        F.datediff(
            "order_delivered_customer_date",
            "order_purchase_timestamp"
        )
    )
    .withColumn(
        "is_late",
        F.col("order_delivered_customer_date") >
        F.col("order_estimated_delivery_date")
    )
)

with_delivery_date = delivered.filter(
    F.col("order_delivered_customer_date").isNotNull()
)
late = with_delivery_date.filter(F.col("is_late"))
```

```
print("Доставленных заказов:", delivered.count())
print("С известной датой доставки:", with_delivery_date.count())
print("Позже ожидаемой даты:", late.count())
```

Результат выполнения

```
Доставленных заказов: 96478
С известной датой доставки: 96470
Позже ожидаемой даты: 7826
```

Из 96 478 заказов со статусом delivered у 96 470 есть фактическая дата доставки. В 7 826 случаях дата позже order_estimated_delivery_date — около 8,11% доставленных заказов с известной датой. Это описание предоставленного набора, а не универсальная оценка качества логистики.

7. Работа с пропусками и преобразованием типов

Пропуск в Spark представлен значением null и его наличие не означает автоматически ошибку. Отсутствующее значение может быть структурно связано с бизнес-состоянием объекта. Например, у отменённого заказа естественно может отсутствовать фактическая дата доставки. Поэтому перед dropna() или fillna() необходимо исследовать распределение пропусков вместе с признаками, объясняющими процесс.

Практический блок 7.1. Подсчёт пропусков во временных полях orders

Для каждого столбца выражение F.col(c).isNull() строит логический признак пропуска. Метод cast("int") переводит True/False в 1/0, а sum() подсчитывает число null.

Список выражений передаётся одному select(), поэтому результатом становится одна строка с отдельным счётчиком для каждого временного поля.

```
null_counts = orders.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c)
    for c in timestamp_cols
])

null_counts.show(vertical=True)
```

Результат выполнения

```
-RECORD 0-----
order_purchase_timestamp      | 0
order_approved_at             | 160
order_delivered_carrier_date   | 1783
order_delivered_customer_date  | 2965
order_estimated_delivery_date  | 0
```

Наиболее заметны 2 965 пропусков `order_delivered_customer_date`. Удалить эти строки сразу было бы ошибкой. Необходимо проверить, связаны ли они с состоянием заказа. Временные поля `purchase` и `estimated delivery` полностью заполнены.

Практический блок 7.2. Предметная проверка пропусков по статусам

Фильтр `isNull()` оставляет только заказы без фактической даты доставки, затем `groupBy("order_status").count()` показывает, к каким состояниям они относятся. Метод `orderBy(desc("count"))` упорядочивает результат по убыванию. Этот шаг превращает проверку качества в предметную диагностику процесса.

```
missing_delivery_by_status = (
  orders
  .filter(F.col("order_delivered_customer_date").isNull())
  .groupBy("order_status")
  .count()
  .orderBy(F.desc("count"))
)
missing_delivery_by_status.show(truncate=False)
```

Таблица 4. Заказы без `order_delivered_customer_date` по статусам

<code>order_status</code>	<code>count</code>
shipped	1 107
canceled	619
unavailable	609
invoiced	314
processing	301
delivered	8
created	5
approved	2

Из 2 965 пропусков фактической даты доставки только 8 относятся к статусу `delivered`. Основная масса приходится на `shipped`, `canceled`, `unavailable` и промежуточные статусы. Следовательно, `null` здесь в значительной степени описывает незавершённый или отменённый процесс, а не случайную потерю данных.

Практический блок 7.3. Заполнение пропуска категориального признака

Для будущего соединения с товарами категориальный пропуск удобно маркировать явным значением `unknown`. В отличие от временной даты, такое

заполнение не создаёт фиктивную числовую величину, оно выделяет отдельную аналитическую категорию.

Таблица `products` читается с `inferSchema=True`, затем `fillna()` применяется только к `product_category_name`. Остальные пропуски физических характеристик не заполняются, поскольку они не нужны в данной задаче и требуют отдельного обоснования.

```
products = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(str(STAGING / "products.csv"))
)

missing_categories = products.filter(
    F.col("product_category_name").isNull()
).count()

products_clean = products.fillna({
    "product_category_name": "unknown"
})

print("Товаров без категории:", missing_categories)
```

Результат выполнения

```
Товаров без категории: 610
```

В `products` у 610 из 32 951 товаров отсутствует `product_category_name`. В аналитике выручки такие товары не следует терять внутренним JOIN. Значение `unknown` позволяет сохранить их как отдельную группу и затем отдельно оценить масштаб неполноты справочника.

8. Агрегации

Агрегация преобразует множество строк в статистики по группам. Метод `groupBy()` задаёт ключ группировки, а `agg()` — набор агрегатных выражений. В распределённом плане группировка часто требует перераспределить строки так, чтобы одинаковые ключи оказались вместе. Поэтому `groupBy()` логически выглядит простой операцией, но в реальном кластере может быть существенно дороже локального фильтра или `select()`.

Практический блок 8.1. Распределение заказов по статусам

Цепочка методов `groupBy("order_status").count()` формирует число заказов для каждого уникального статуса. Метод `orderBy()` нужен только для удобства

чтения. Эта агрегация одновременно проверяет кардинальность категориального признака и даёт контекст для интерпретации пропусков временных полей.

```
status_stats = (
    orders
    .groupBy("order_status")
    .count()
    .orderBy(F.desc("count"))
)

status_stats.show(truncate=False)
```

Таблица 5. Число заказов по статусам

order_status	count
delivered	96 478
shipped	1 107
canceled	625
unavailable	609
invoiced	314
processing	301
created	5
approved	2

Статус delivered доминирует имея 96 478 из 99 441 заказов. Именно поэтому общая статистика времени доставки в основном описывает завершённые операции, а редкие статусы требуют осторожности при процентных сравнениях.

Практический блок 8.2. Несколько агрегатных функций для способов оплаты

Атрибут order_payments читается с автоматическим выводом числовых типов. В agg() одновременно вычисляются число строк, суммарная стоимость и среднее значение платежа. F.count("*") считает строки, F.sum() и F.avg() работают с payment_value, F.round() ограничивает число знаков. Именованное через alias() превращает технические выражения в понятные поля итоговой таблицы.

```
payments = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(str(STAGING / "order_payments.csv"))
)

payment_stats = (
```

```

payments
.groupBy("payment_type")
.agg(
  F.count("*").alias("payments"),
  F.round(F.sum("payment_value"), 2).alias("total_value"),
  F.round(F.avg("payment_value"), 2).alias("avg_value")
)
.orderBy(F.desc("total_value"))
)

payment_stats.show(truncate=False)

```

Таблица 6. Агрегаты по типу платежа

payment_type	payments	total_value	avg_value
credit_card	76 795	12 542 084.19	163.32
boleto	19 784	2 869 361.27	145.03
voucher	5 775	379 436.87	65.70
debit_card	1 529	217 989.79	142.57
not_defined	3	0.00	0.00

По числу строк и суммарной стоимости доминируют платежи credit_card: 76 795 записей и около 12,54 млн единиц payment_value. Поле payment_sequential означает, что одна покупка может иметь несколько платёжных строк, поэтому COUNT(*) в этой таблице нельзя автоматически интерпретировать как число заказов.

9. Соединение DataFrame

Метод join() объединяет строки по ключу. INNER JOIN оставляет только совпавшие ключи с обеих сторон. LEFT JOIN сохраняет все строки левого DataFrame и добавляет null там, где справочник не дал совпадения.

Для аналитической витрины позиций заказов LEFT JOIN часто безопаснее, так как он позволяет измерить неполноту справочника вместо потери транзакций. После нескольких JOIN особенно важно контролировать число строк и неоднозначные имена столбцов.

Практический блок 9.1. Позиции заказов + товары + перевод категорий

Сначала читается маленький справочник переводов. alias() даёт короткие имена DataFrame, а условие F.col("i.product_id") == F.col("p.product_id") явно задаёт ключ первого LEFT JOIN.

Второй LEFT JOIN связывает исходное название категории со справочником перевода. Метод select() оставляет поля факта и категорий. Метод coalesce() выбирает первое непустое значение английского перевода, далее

исходное название, затем literal "unknown". Таким образом, даже неполные справочники не удаляют позиции заказа.

```
translations = (
    spark.read
    .option("header", True)
    .csv(str(STAGING / "product_category_name_translation.csv"))
)

sales = (
    items.alias("i")
    .join(
        products_clean.select(
            "product_id", "product_category_name"
        ).alias("p"),
        F.col("i.product_id") == F.col("p.product_id"),
        "left"
    )
    .join(
        translations.alias("t"),
        F.col("p.product_category_name") ==
        F.col("t.product_category_name"),
        "left"
    )
    .select(
        F.col("i.order_id").alias("order_id"),
        F.col("i.order_item_id").alias("order_item_id"),
        F.col("i.product_id").alias("product_id"),
        F.col("i.price").alias("price"),
        F.col("i.freight_value").alias("freight_value"),
        F.coalesce(
            F.col("t.product_category_name_english"),
            F.col("p.product_category_name"),
            F.lit("unknown")
        ).alias("category")
    )
)

print("Строк до JOIN:", items.count())
print("Строк после JOIN:", sales.count())
print(
    "Позиций категории unknown:",
    sales.filter(F.col("category") == "unknown").count()
)
```

Результат выполнения

```
Строк до JOIN: 112650
Строк после JOIN: 112650
Позиций категории unknown: 1603
```

LEFT JOIN сохранил все 112 650 позиций заказа. Для 1 603 позиций категория осталась unknown. Контроль числа строк после JOIN является обязательной проверкой: неожиданный рост мог бы означать связь многие-ко-многим, а уменьшение при INNER JOIN — потерю фактов из-за неполного справочника.

Практический блок 9.2. Заказы и клиенты, аналитика просрочки по штатам

Сущность customers содержит географическую категорию customer_state. После чтения CSV с inferSchema=True к delivered добавляется штат клиента по customer_id. Затем данные с известной датой доставки агрегируются: count() даёт число доставленных заказов, sum(when(is_late, 1).otherwise(0)) — число просроченных, а отношение двух агрегатов формирует процент. Классически join() является промежуточным этапом перед аналитической агрегацией.

```
customers = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(str(STAGING / "customers.csv"))
)

late_by_state = (
    delivered
    .join(
        customers.select("customer_id", "customer_state"),
        on="customer_id",
        how="left"
    )
    .filter(F.col("order_delivered_customer_date").isNotNull())
    .groupBy("customer_state")
    .agg(
        F.count("*").alias("delivered"),
        F.sum(
            F.when(F.col("is_late"), 1).otherwise(0)
        ).alias("late")
    )
    .withColumn(
        "late_pct",
        F.round(F.col("late") / F.col("delivered") * 100, 2)
    )
    .orderBy(F.desc("late_pct"))
)

late_by_state.show(10, truncate=False)
```

Таблица 7. Десять штатов с наибольшей долей поздней доставки в предоставленных данных

state	delivered	late	late_pct, %
AL	397	95	23.93
MA	717	141	19.67
PI	476	76	15.97
CE	1 279	196	15.32
SE	335	51	15.22
BA	3 256	457	14.04
RJ	12 350	1 664	13.47
TO	274	35	12.77
PA	946	117	12.37
ES	1 995	244	12.23

В предоставленном наборе максимальная доля доставок позже estimated date наблюдается для AL — 23,93%, но там только 397 доставленных заказов. Проценты нужно интерпретировать вместе с размером группы: результат описывает выборку и не доказывает причинное влияние штата на логистику.

10. Spark SQL

Spark SQL предоставляет декларативный интерфейс поверх тех же DataFrame и того же оптимизатора. Метод `createOrReplaceTempView()` регистрирует DataFrame как временное представление только внутри текущей SparkSession. Метод `spark.sql()` принимает SQL-запрос и возвращает новый DataFrame. DataFrame API и SQL не являются двумя разными движками, они формируют планы одного Spark SQL слоя.

Практический блок 10.1. Выручка по товарным категориям через SQL

Набор данных Spark DataFrame `sales` регистрируется под именем `sales` для использования его в SparkSQL запросе. SQL агрегирует число позиций, число уникальных заказов, сумму `price` и среднюю цену по `category`. `COUNT(DISTINCT order_id)` принципиально отличается от `COUNT(*)`, так как один заказ может содержать несколько строк `order_items`. Строчка `ORDER BY revenue DESC LIMIT 10` формирует компактный рейтинг.

```
Sales.createOrReplaceTempView("sales")

category_sql = spark.sql("""
SELECT
    category,
    COUNT(*) AS items,
    COUNT(DISTINCT order_id) AS orders,
    ROUND(SUM(price), 2) AS revenue,
```

```

ROUND(AVG(price), 2) AS avg_price
FROM sales
GROUP BY category
ORDER BY revenue DESC
LIMIT 10
""")

```

```
category_sql.show(truncate=False)
```

Таблица 8. Топ-10 категорий по сумме price

category	items	orders	revenue	avg_price
health_beauty	9 670	8 836	1 258 681.34	130.16
watches_gifts	5 991	5 624	1 205 005.68	201.14
bed_bath_table	11 115	9 417	1 036 988.68	93.30
sports_leisure	8 641	7 720	988 048.97	114.34
computers_accessories	7 827	6 689	911 954.32	116.51
furniture_decor	8 334	6 449	729 762.49	87.56
cool_stuff	3 796	3 632	635 290.85	167.36
housewares	6 964	5 884	632 248.66	90.79
auto	4 235	3 897	592 720.11	139.96
garden_tools	4 347	3 518	485 256.46	111.63

Наибольшая сумма price в сформированной витрине относится к health_beauty — около 1,259 млн, затем watches_gifts — около 1,205 млн и bed_bath_table — около 1,037 млн. Это сумма цены товарных позиций, а не полная платёжная выручка: freight_value и возможные особенности платежей здесь не включены.

Практический блок 10.2. Эквивалентная агрегация через DataFrame API

Для сравнения та же логика формируется через groupBy().agg(). Функция countDistinct() соответствует COUNT(DISTINCT ...). Результаты SQL и DataFrame API должны совпасть, поскольку оба выражения передаются одному оптимизатору Spark.

```

category_sales = (
    sales
    .groupBy("category")
    .agg(
        F.count("*").alias("items"),
        F.countDistinct("order_id").alias("orders"),
        F.round(F.sum("price"), 2).alias("revenue"),
        F.round(F.avg("price"), 2).alias("avg_price")
    )
)

print("Категорий в витрине:", category_sales.count())

category_sales.orderBy(

```

```
F.desc("revenue")
).show(10, truncate=False)
```

Результат выполнения

Категорий в витрине: 74

После объединения переведённых, непереведённых и unknown-категорий итоговая витрина содержит 74 группы. Первые строки по revenue должны совпасть с SQL-запросом выше. Совпадение является простой проверкой эквивалентности двух способов построения аналитики.

11. Запись результата

Spark записывает распределённый DataFrame не как один файл, а как каталог с несколькими part-файлами и служебными объектами. Это соответствует параллельной природе вычислений. Разные разделы могут записываться независимо. Parquet сохраняет схему и колоночную организацию, поэтому предпочтителен для последующих аналитических операций. CSV остаётся удобным обменным форматом, но типы при повторном чтении нужно восстанавливать.

Практический блок 11.1. Сохранение аналитической витрины в Parquet и CSV

Цепочка `write.mode("overwrite")` позволяет повторно запускать notebook, заменяя предыдущий каталог результата. Метод `parquet()` создаёт колоночный набор файлов. Для CSV показан `coalesce(1)`, поскольку итоговая таблица имеет всего 74 строки и удобно получить один part-файл для отчёта. На больших фактах `coalesce(1)` использовать не следует, оно сводит запись к одному разделу и создаёт узкое место.

```
OUT = Path("spark_output")
OUT.mkdir(exist_ok=True)

(
    category_sales
    .write
    .mode("overwrite")
    .parquet(str(OUT / "category_sales_parquet"))
)

(
    category_sales
    .coalesce(1)
    .write

```



```

        .mode("overwrite")
        .option("header", True)
        .csv(str(OUT / "category_sales_csv"))
    )

category_back = spark.read.parquet(
    str(OUT / "category_sales_parquet")
)

print("Строк после round-trip:", category_back.count())
category_back.printSchema()

```

Результат выполнения

```

Строк после round-trip: 74
root
 |-- category: string (nullable = true)
 |-- items: long (nullable = true)
 |-- orders: long (nullable = true)
 |-- revenue: double (nullable = true)
 |-- avg_price: double (nullable = true)

```

Parquet-результат содержит те же 74 агрегированных категории и хранит числовую схему. В отличие от CSV, при чтении Parquet не требуется inferSchema. Каталоги spark_output должны сохраняться вместе с notebook, если они входят в сдаваемый результат.

Практический блок 11.2. Корректное завершение SparkSession

Метод spark.stop() освобождает ресурсы локального SparkContext и JVM. В интерактивном notebook его выполняют после последнего блока, а при повторном запуске всей работы новая SparkSession будет создана заново. Такой финал полезен для воспроизводимости и особенно важен, если на одном компьютере последовательно запускается несколько Spark-приложений.

```
spark.stop()
```

После stop() текущие DataFrame больше нельзя вычислять через завершённую сессию. Поэтому блок остановки должен оставаться последней выполняемой ячейкой учебного pipeline.

12. Самостоятельная работа. Распределённый анализ собственного датасета

После выполнения учебной последовательности необходимо воспроизвести тот же подход на собственном наборе данных. Цель самостоятельной части — показать, что студент умеет переносить модель Spark на другую предметную область, а не только повторять команды для ecommerce.sqlite. Предпочтителен

источник с несколькими связанными таблицами или с таблицей фактов и справочником.

12.1. Минимальные требования к данным

- не менее 20 000 строк суммарно и не менее 6 информативных полей в основной таблице;
- не менее двух числовых и двух категориальных признаков;
- минимум два логически связанных DataFrame для выполнения содержательного JOIN; допускается факт + справочник;
- желательно наличие пропусков или строковых полей, требующих преобразования типов;
- источник и условия использования данных должны быть явно указаны;
- не использовать персональные или иные данные, распространение которых нарушает требования конфиденциальности.

12.2. Обязательная последовательность выполнения

1. Подготовить источник к чтению Spark: CSV/JSON/Parquet либо воспроизводимый staging-этап из исходной СУБД. Объяснить, почему выбран именно этот способ.
2. Создать SparkSession в local[*], зафиксировать версии Python, Java и Spark, а также основные параметры сессии.
3. Прочитать основную таблицу и вывести число строк, список столбцов и printSchema().
4. Для одного DataFrame задать явный StructType. Сравнить явную схему с inferSchema или чтением строками.
5. Выполнить не менее пяти разных transformations, включая select(), filter() и withColumn(). Создать минимум один содержательный вычисляемый признак.
6. Посчитать пропуски минимум в трёх полях. Для каждого принятого fillna/dropna/filter объяснить предметную причину; запрещено удалять строки только потому, что в них есть null.
7. Выполнить минимум две группировки. Одна должна использовать не менее трёх агрегатных функций.
8. Выполнить минимум один LEFT JOIN и один INNER JOIN либо обоснованно объяснить, почему второй тип соединения не подходит. Сравнить число строк до и после JOIN.
9. Создать temporary view и выполнить не менее двух Spark SQL запросов, один из которых содержит GROUP BY и сортировку по агрегату.
10. Сформировать итоговый аналитический DataFrame и записать его в Parquet. Для небольшого агрегата дополнительно сохранить CSV.

11. Повторно прочитать Parquet, сравнить число строк и схему с объектом до записи.
12. После каждого существенного блока сформулировать краткий вывод о собственных данных, а в конце — не менее пяти итоговых выводов и двух ограничений анализа.

12.3. Что должно быть в отчёте

Таблица 9. Рекомендуемая структура отчёта по самостоятельной работе

Раздел	Содержание
1. Постановка задачи	Источник данных, предметная область, цель распределённой обработки.
2. Архитектура входа	Исходный формат, staging при необходимости, схема связанных DataFrame.
3. SparkSession и схема	Версии, master, printSchema(), явный StructType.
4. Transformations	Код, вычисляемые признаки, число строк после фильтров.
5. Качество данных	Пропуски, преобразование типов, обоснование действий.
6. Агрегации и JOIN	Таблицы результатов, контроль строк и интерпретация.
7. Spark SQL	SQL-запросы и сопоставление с DataFrame API.
8. Экспорт	Parquet/CSV, round-trip, схема итоговой витрины.
9. Итоги	Не менее 5 выводов и 2 ограничений.

12.4. Что необходимо предоставить к сдаче

1. Jupyter Notebook с выполненными учебными блоками по ecommerce.sqlite; notebook должен выполняться командой Run All после перезапуска ядра.
2. Отдельный Notebook либо самостоятельный раздел того же Notebook с анализом собственного набора данных.
3. Каталог итогового Parquet и при необходимости CSV-результат самостоятельной работы.
4. Краткие выводы после практических блоков и итоговые выводы по двум наборам данных.

13. Контрольные вопросы

1. Чем local[*] отличается от физического Spark-кластера и какие элементы программной модели при этом сохраняются?
2. Каковы роли Driver и Executor в Spark-приложении?
3. Что такое partition и почему число partitions влияет на производительность?
4. В чём различие transformation и action? Приведите по три примера каждого типа.
5. Что означает lazy evaluation и почему время выполнения withColumn() без action не отражает стоимость вычислений?

6. Почему CSV не содержит строгой аналитической схемы и в каких случаях явный StructType предпочтительнее inferSchema?
7. Почему строку ISO-даты следует преобразовать в timestamp, даже если лексикографическое сравнение иногда даёт правильный порядок?
8. Почему наличие null не означает, что строку нужно удалить? Как это показал пример order_delivered_customer_date?
9. Чем select() отличается от filter(), а withColumn() — от простого выбора столбца?
10. Почему после JOIN необходимо сравнивать число строк до и после операции?
11. В чём содержательное различие INNER JOIN и LEFT JOIN при построении фактовой аналитической витрины?
12. Почему COUNT(*) и COUNT(DISTINCT order_id) дают разные бизнес-смыслы для order_items?
13. Как связаны Spark SQL и DataFrame API? Являются ли они разными вычислительными движками?
14. Почему Spark записывает DataFrame в каталог с part-файлами, а не обязательно в один файл?
15. Почему coalesce(1) допустим для маленькой 74-строчной витрины, но является плохой общей стратегией записи больших данных?
16. Какие преимущества Parquet даёт последующим Spark-задачам по сравнению с CSV?

Перед сдачей необходимо перезапустить ядро Jupyter и выполнить Run All. Работа считается воспроизводимой только если staging-каталог, SparkSession, DataFrame, JOIN, SQL и экспорт создаются последовательно без зависимости от объектов, оставшихся в памяти после ручного выполнения отдельных ячеек.